

Projetos em Ambientes Web e Cloud

Licenciatura em Tecnologias Digitais e
Inteligência Artificial

Aula 7

João Matos

joao_miguel_goncalves_matos@iscte-iul.pt

2º Semestre

useEffect

- Hook que permite configurar determinados comportamentos a partir de mudanças no estado do componente;
- Recebe um callback que pode ser “chamado” quando, por exemplo:
 - O componente é criado; alterado ou removido.

```
useEffect(() => {  
    console.log("componente criado")  
}, [variavel1, variavel2]);
```

- As variáveis permitem dizer ao *useEffect* para correr quando algum destes estados é alterado

useEffect

- Permite chamar funções dentro do componente:
 - Fetch data; alterar estado; modificar a DOM

```
useEffect(() => {  
  console.log("componente criado")  
}, []);
```

```
useEffect(() => {  
  console.log("componente criado")  
  return function () {  
    console.log("componente removido")  
  }  
}, []);
```

```
useEffect(() => {  
  console.log("componente atualizado")  
});
```

```
useEffect(() => {  
  console.log("estado atualizado")  
}, [estado]);
```

```
useEffect(() => {  
  // Code that requires cleanup...  
  return () => {  
    // Cleanup code...  
  };  
}, [variavel]);
```

React Router (useParams)

```
import { BrowserRouter as Router, Routes, Route } from
"react-router-dom";

import Home from "../pages/Home";
import Products from "../pages/Products";
import ProductPage from "../pages/ProductPage";

function App() {
  return (
    <div className="App">
      <Router>
        <Header />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/products" element={<Products />} />
          <Route path="/product/:product_id" element={<ProductPage />}
        />
        </Routes>
      </Router>
    </div>
  );
}
```

ProductPage:

```
import { useParams } from "react-router-dom";

const ProductPage = () => {

  useEffect(() => {
    let { product_id } = useParams;
  }, []);

  return (
    ( ... )
  )
}
```

React Router (useLocation)

`http://localhost:3000/products?author="john"&category="mongodb"`

pathname / endpoint

search

```
import { useLocation } from "react-router-dom";

const ProductPage = () => {
  const location = useLocation();

  console.log(location);

  let author = new URLSearchParams(location.search).get('author');
  console.log(author);

  return (
    ( ... )
  )
}
```

useLocation: permite aceder ao objeto de localização do navegador. Este objeto contém informações sobre o URL atual.

```
const location = useLocation();
```

```
pathname: "/books"
search: "?author=%22john%22&category=%22mongodb%22"
state: null
```

React Router (useLocation)

`http://localhost:3000/products?author="john"&category="mongodb"`

pathname / endpoint

search

```
import { useLocation } from "react-router-dom";

const ProductPage = () => {
  const location = useLocation();

  console.log(location);

  let author = new URLSearchParams(location.search).get('author');
  console.log(author);

  return (
    ( ... )
  )
}
```

URLSearchParams: método Javascript para manipular os parâmetros de um URL:

- `get(name)`: retorna o valor do parâmetro passado;
- `getAll(name)`: retorna todos os valores associados ao parâmetro passado;
- `has(name)`: verifica se o parâmetro existe no URL

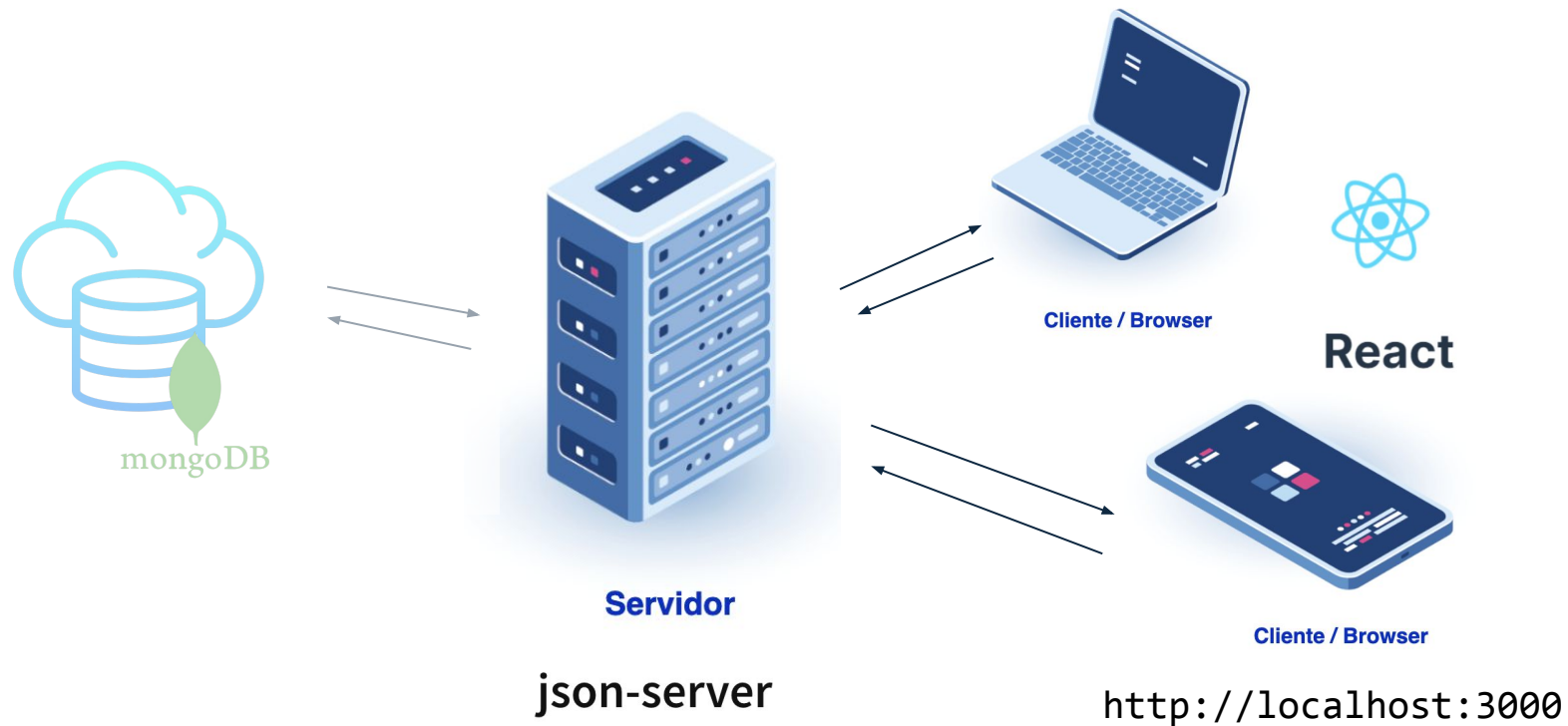
React Router (useLocation)

- O método Link do React Router permite enviar dados através do *state*
 - Neste exemplo, a página “Products” consegue aceder ao state passado pelo componente anterior que fez o Link.
- > {product_id: 10, name: 'Produto'}

```
<Link  
  to="/books"  
  state={{product_id: 10, name: "Produto"}}  
>Books</Link>
```

```
import { useLocation } from "react-router-dom";  
  
const ProductPage = () => {  
  const location = useLocation();  
  
  const { state } = location;  
  console.log(state);  
  
  return (  
    ( ... )  
  )  
}
```

Full-Stack



`http://localhost:3030/products`

`http://localhost:3030/favorites`

Backend Server

json-server

Module que vai permitir simular um servidor para interagir com a base de dados;

Permite o uso de métodos HTTP (GET, POST, PUT, DELETE)

Fácil acesso a objetos JSON

Simula API localmente

Instalação:

```
npm install json-server
```

Run:

```
json-server --watch db.json --port 3030
```

API em <http://localhost:3030/>

```
GET    /products
GET    /products/:id
POST   /products
PUT    /products/:id
PATCH /products/:id
DELETE /products/:id
```

Backend Server

json-server: aceder/manipular dados

GET /products - lista de todos os produtos

GET /products/1 - retorna informação do produto com id = 1

POST /products - cria uma nova casa

PUT /products/1 - Actualiza os dados da casa com id = 1

DELETE /products/1 - remove casa da base de dados com id = 1

Filtros

GET /products?price=2000

Retorna todos os produtos com preço igual a 2000

GET /products?price_gt=1000&price_lt=2000

Operadores:

_lt → <

_lte → <=

_gt → >

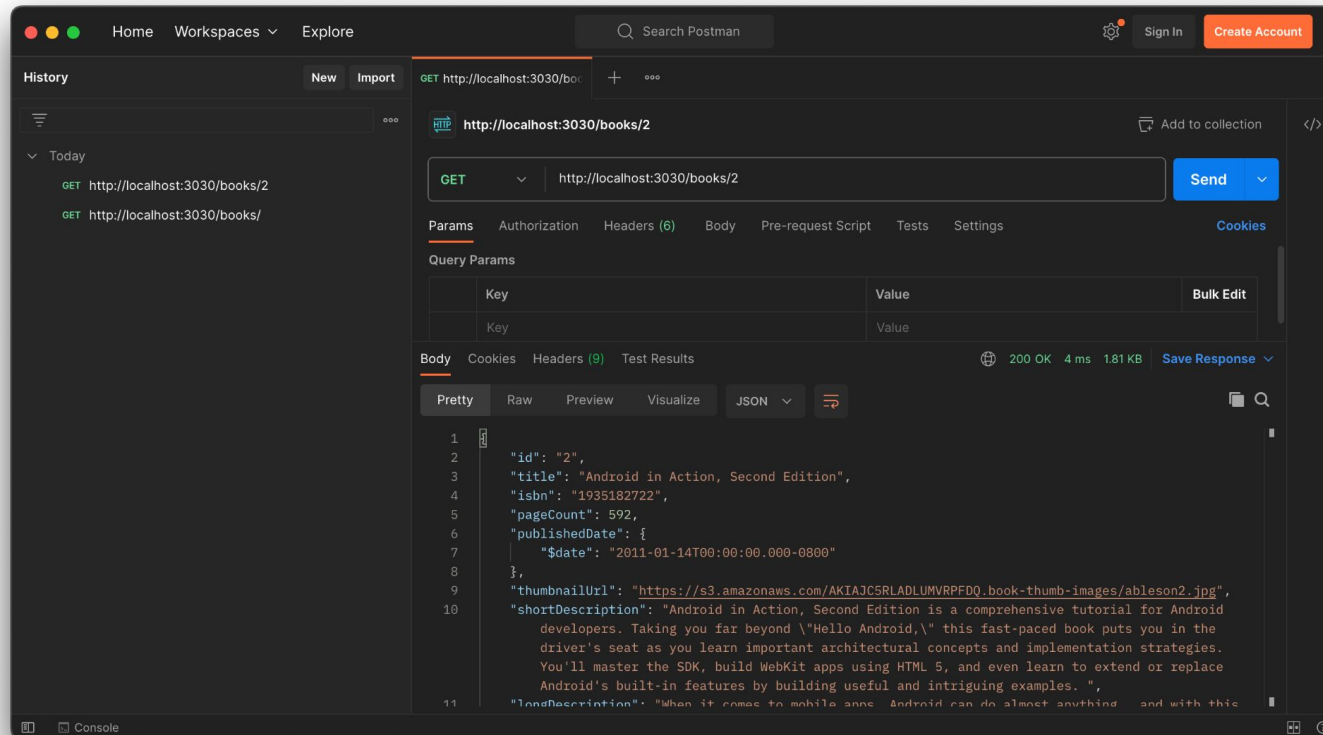
_gte → >=

_ne → !=

GET /products?_sort=price

Lista de todos os produtos, ordenadas por preço

json-server



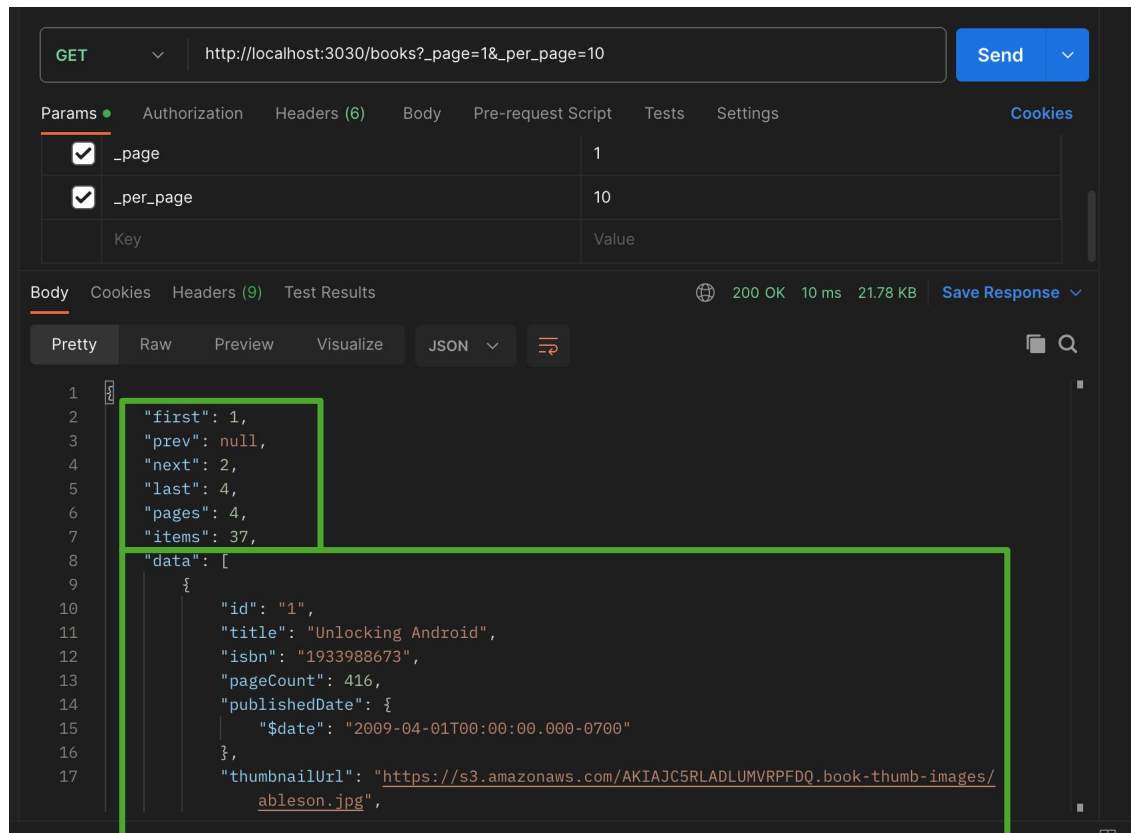
GET /products
GET /products/:id
POST /products
PUT /products/:id
PATCH /products/:id
DELETE /products/:id

Conditions

lt → <
lte → <=
gt → >
gte → >=
ne → !=

Exemplo: ...?score_gt=4

json-server



GET /products?_page=1&_per_page=25

Permite fazer o *request* por página. A resposta inclui:

- array de objetos “data” com a lista de livros;
- informação referente a todas as páginas:
 - “first”: primeira página;
 - “prev”: página anterior;
 - “next”: página seguinte;
 - “last”: última página;
 - “pages”: número total de páginas;
 - “items”: número de objetos (livros)

Fetch API

- Método Javascript para fazer *requests* de recursos (documentos, imagens, dados, etc.) através da rede.
- Usado principalmente para fazer chamadas HTTP assíncronas.
- `json()`: extrai os dados JSON ao objeto *Response*
- Se ocorrer no *request*, a promessa é rejeitada e o bloco 'catch' é executado

Syntax:

```
fetch('https://url_exemplo.com/products')  
  .then(response => {  
    if (!response.ok) {  
      throw new Error('Erro ao carregar os dados');  
    }  
    return response.json();  
  })  
  .then(data => {  
    console.log(data);  
  })  
  .catch(error => {  
    console.error('Ocorreu um erro:', error);  
  });
```

- Para fazer pesquisas com diferentes campos, basta adicionar esses valores à query string (URL)

`http://localhost:3030/products?category="informatica"`

Fetch API: exemplos

Método POST:

```
fetch('https://url_exemplo.com/products', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    title: 'New Product'  
  })  
})  
  .then(response => {  
    if (!response.ok) {  
      throw new Error('Erro ao enviar os dados');  
    }  
    return response.json();  
  })  
  .then(data => {  
    console.log('POST:', data);  
  })  
  .catch(error => {  
    console.error('Ocorreu um erro:', error);  
  });
```

'Content-Type': 'application/json'

- Indica ao servidor que o corpo da mensagem ou do request está no formato JSON

body: JSON.stringify({})

- Converte o objeto Javascript em uma string JSON

method

- POST | PUT | DELETE

Javascript: callbacks

Callback: funções passadas como argumento a outra função.

São executadas mais tarde, e geralmente após algum tipo de evento ou operação assíncrona.

```
function funcaoA(funcaoB) {  
    funcaoB();  
}  
  
function funcaoB() {  
    console.log("Função callback!!")  
}
```

Javascript: Promises

- Uma Promise é um objeto que representa a eventual conclusão ou falha de uma operação assíncrona.
- Em vez de passarmos callbacks a uma função, podemos utilizar uma promise para fazer essa gestão.
- Uma Promise tem 3 estados diferentes:
 - Pendente
 - Realizada
 - Rejeitada

```
new Promise(function(myResolve, myReject) {  
  if (tudoOK) {  
    myResolve(resultado);  
  } else {  
    myReject(erro);  
  }  
}).then((resultado) => {  
  console.log(resultado)  
}).catch((erro) => {  
  console.log(erro)  
})
```


Javascript: Promises

- Quando é executada a função “myResolve” é marcada como realizada e os callbacks registados com o then() são executados pela ordem com que foram definidos.
- Quando é executada a função “myReject”, a Promise é marcada como rejeitada e o bloco registado com o catch será executado.
- Os valores que entram no **then** e no **catch** são os valores que passarmos às funções myResolve e myReject, respetivamente.

```
new Promise(function(myResolve, myReject) {  
  if (tudoOK) {  
    myResolve(resultado);  
  } else {  
    myReject(erro);  
  }  
}).then((resultado) => {  
  console.log(resultado)  
}).catch((erro) => {  
  console.log(erro)  
})
```

Javascript: Funções assíncronas

Funções que são executadas em paralelo são chamadas funções assíncronas;

Para sabermos se um processo assíncrono terminou podemos usar um *Callback*

```
let funcaoPromise = new Promise((resolve, reject) => {  
    setTimeout(() => {  
        resolve('Operação bem-sucedida!');  
    }, 2000);  
});  
  
funcaoPromise.then((resultado) => {  
    console.log(resultado);  
}).catch((erro) => {  
    console.log(erro);  
});
```

Javascript: async ... await

“async and await make promises easier to write”

async: faz com que a função retorne uma Promise

await: faz a função esperar por uma Promise;

Apenas pode ser usada dentro de uma função async; faz com que a função espere pela Promise estar resolvida.

```
async function minhaFuncao() {  
  const resultado = await new Promise(function(myResolve, myReject) {  
    if (tudoOK) {  
      myResolve(resultado);  
    } else {  
      myReject(erro);  
    }  
  })  
  
  console.log(resultado);  
}
```

Javascript: async ... await

- O resultado passado no “myResolve” da promise é o retorno da instrução await

- O que acontece se for feito o “reject” na promise?

O await vai provocar um lançamento de exceção que pode ser tratada por um block try/catch

```
async function minhaFuncao() {  
  try {  
    const resultado = await new Promise(function(myResolve, myReject) {  
      if (tudoOK) {  
        myResolve(resultado);  
      } else {  
        myReject(erro);  
      }  
    });  
    console.log(resultado);  
  } catch (erro) {  
    console.log(erro);  
  }  
}
```

async...await

```
const [data, setData] = useState([]);

const fetchData = async () => {
  try {
    const response = await fetch('https://API_URL', {
      method: 'GET',
      headers: {
        '...'
      }
    });

    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    const data = await response.json();
    setData(data);
  } catch (error) {
    console.error('Error:', error);
  }
}

useEffect(() => {
  fetchData();
}, []);
```

React: useContext

- *Manage State Globally*
- Passar dados entre vários componentes que estão vários níveis abaixo da árvore, sem ter de passar props entre vários componentes;
- Simplifica o acesso a dados que são compartilhados por diferentes componentes;

Para criar um “*Contexto global*”, precisamos de o criar no componente do nível mais alto, **createContext**:

```
import { useState, createContext } from "react";

export const CartContext = createContext()

function App() {

  return (
    ( ... )
  )
}
```

React: useContext

Passar dados (global state) a todos os componentes

- App.js: cria o state “cart” que é passado a todos os componentes.
- **useContext**: permite aceder aos dados criados no *Context Provider*

```
import { useEffect, useState, useContext } from "react";
import { CartContext } from '../App';

function Home() {
  const cart = useContext(CartContext);

  return (
    <div className="home">
      {cart}
    </div>
  )
}
```

```
import { useState, createContext } from "react";

(...)

export const CartContext = createContext();

function App() {
  const [cart, setCart] = useState([]);

  return (
    <div className="App">
      <CartContext.Provider value={cart}>
        <Router>
          <Header />
          <Routes>
            <Route exact path="/" element={<Home />} />
            <Route path="/books" element={<Books />} />
            <Route path="/authors" element={<Authors />} />
          </Routes>
        </Router>
      </CartContext.Provider>
    </div>
  );
}

export default App;
```

React: useContext

Atualizar dados passados globalmente

```
(...)  
function App() {  
  const [cart, setCart] = useState(20);  
  
  return (  
    <div className="App">  
      <CartContext.Provider value={{cart, setCart}}>  
        <Router>  
          <Routes>  
            <Route exact path="/" element={<Home />} />  
          </Routes>  
        </Router>  
      </CartContext.Provider>  
    </div>  
  );  
}  
(...)
```

```
import { useEffect, useState, useContext } from "react";  
import { CartContext } from '../App';  
  
function Home() {  
  const {cart, setCart} = useContext(CartContext);  
  
  function atualizarContext() {  
    setCart(cart + 1)  
  }  
  
  return (  
    <div className="home">  
      {cart}  
      <button onClick={atualizarContext}>  
        Atualizar Cart  
      </button>  
    </div>  
  )  
}
```


React: useContext

Estado inicial, exemplo:

```
const [cart, setCart] = useState({
  books: [],
  total: 0,
  quantidade: 0
});

const adicionarAoCarrinho = (livro) => {
  setCart([
    ...cart,
    { ...livro, quantidade: 1 }
  ]);
};
```

Muitas vezes precisamos de saber o valor anterior do estado antes de fazer o update:

```
const [valor, setValor] = useState(0);

const updateState = () => {
  setValor(prevState => prevState + 1);
};
```

React: useContext

Método map() dentro do setState:

```
const [cart, setCart] = useState([]);

const adicionarAoCarrinho = (livro) => {
  setCart(
    cart.map((item) => {
      if(item.id === livro.id) {
        return { ...item, quantity: item.quantidade + 1 }
      } else {
        return item
      }
    })
  )
};
```



**Build fast, responsive sites
with Bootstrap**

- Front-end Framework que facilita a implementação de páginas web, com estilos CSS pré-definidos e diferentes componentes.
- Facilita a criação de layouts e de páginas Web Responsive

```
npm install react-bootstrap
```

Exercício

Casas

[Início](#) [Sobre](#) [Serviços](#) [Contato](#)

Lista de casas

Filtros: Arrendar

FOR SALE

Vila in Sesimbra

Sesimbra, Portugal

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.

Price

\$1000

Consultar

© 2024 Casas. Todos os direitos reservados.

Componente Header

Componente HouseCard

Componente Footer